

Tarea N5

Nombre : Joel Velez

Link del repositorio: <https://github.com/Joe-575/Tarea-N5>

Ejercicios

1. Sea $f(x) = -x^3 - \cos(x)$ y $p_0 = -1$. Use el método de Newton y de la Secante para encontrar p_2 . ¿Se podría usar $p_0 = 0$?

El método de newton utiliza la siguiente fórmula iterativa

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}$$

Su derivada es: $f'(x) = -3x^2 + \sin(x)$

Con el valor $p_0 = 0$

$$f'(0) = -3(0)^2 + \sin(0) = 0 + 0 = 0$$

```
import numpy as np
from scipy import optimize

# función y su derivada
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return -x**3 - np.cos(x)

def df(x):
    return -3*x**2 + np.sin(x)

p0 = -1.0

print(" Método de Newton ")
p2_newton, info_newton = optimize.newton(f, x0=p0, fprime=df, maxiter=2, full_output=True, disp=False)
print(f" El valor de p2: {p2_newton:.6f}")

print("\n Método de la Secante ")
# La secante estricta necesita x0 y x1.
# calcularemos x1 con un paso de Newton.
p1 = p0 - f(p0) / df(p0)
p2_secante, info_secante = optimize.newton(f, x0=p0, x1=p1, maxiter=1, full_output=True, disp=False)
print(f" El valor de p2: {p2_secante:.6f}")
```

Método de Newton

```
-> Iteración evaluando en p = -1.000000  
-> Iteración evaluando en p = -0.880333  
El valor de p2: -0.865684
```

Método de la Secante

```
-> Iteración evaluando en p = -1.000000  
-> Iteración evaluando en p = -1.000000  
-> Iteración evaluando en p = -0.880333  
-> Iteración evaluando en p = -0.867235  
El valor de p2: -0.867235
```

2. Encuentre soluciones precisas dentro de 10^{-4} para los siguientes problemas:

a. $x^3 - 2x^2 - 5 = 0, [1, 4]$

b. $x^3 + 3x^2 - 1 = 0, [-3, -2]$

c. $x - \cos(x) = 0, [0, \frac{\pi}{2}]$

d. $x - 0.8 - 0.2\sin(x) = 0, [0, \frac{\pi}{2}]$

```
# función y su derivada del ejercicio 2. literal a)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return x**3 - 2*x**2 - 5

def df(x):
    return 3*x**2 - 4*x

# definición de punto medio
a = 1.0
b = 4.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")
raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-4)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-4)
print(f" El valor final es: {raiz_secante:.6f}")
```

Método de Newton

```
-> Iteración evaluando en p = 2.500000  
-> Iteración evaluando en p = 2.714286  
-> Iteración evaluando en p = 2.690952  
-> Iteración evaluando en p = 2.690647  
El valor final es: 2.690647
```

Método de la Secante

```

-> Iteración evaluando en p = 1.000000
-> Iteración evaluando en p = 4.000000
-> Iteración evaluando en p = 1.545455
-> Iteración evaluando en p = 1.996934
-> Iteración evaluando en p = 4.105063
-> Iteración evaluando en p = 2.294699
-> Iteración evaluando en p = 2.478726
-> Iteración evaluando en p = 2.751368
-> Iteración evaluando en p = 2.683084
-> Iteración evaluando en p = 2.690398
-> Iteración evaluando en p = 2.690648
El valor final es: 2.690647

```

```

# función y su derivada del ejercicio 2. literal b)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return x**3 + 3*x**2 - 1

def df(x):
    return 3*x**2 + 6*x

# definición de punto medio
a = -3.0
b = -2.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")
raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-4)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-4)
print(f" El valor final es: {raiz_secante:.6f}")

```

Método de Newton

```

-> Iteración evaluando en p = -2.500000
-> Iteración evaluando en p = -3.066667
-> Iteración evaluando en p = -2.900876
-> Iteración evaluando en p = -2.879720
-> Iteración evaluando en p = -2.879385
El valor final es: -2.879385

```

Método de la Secante

```

-> Iteración evaluando en p = -3.000000
-> Iteración evaluando en p = -2.000000
-> Iteración evaluando en p = -2.750000
-> Iteración evaluando en p = -3.066667
-> Iteración evaluando en p = -2.862024
-> Iteración evaluando en p = -2.877186
-> Iteración evaluando en p = -2.879414
El valor final es: -2.879385

```

```

# función y su derivada del ejercicio 2. literal c)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return x - np.cos(x)

def df(x):
    return 1 + np.sin(x)

# definición de punto medio
a = 0.0
b = np.pi / 2.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")
raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-4)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-4)
print(f" El valor final es: {raiz_secante:.6f}")

```

Método de Newton

```

-> Iteración evaluando en p = 0.785398
-> Iteración evaluando en p = 0.739536
-> Iteración evaluando en p = 0.739085
El valor final es: 0.739085

```

Método de la Secante

```

-> Iteración evaluando en p = 0.000000
-> Iteración evaluando en p = 1.570796
-> Iteración evaluando en p = 0.611015
-> Iteración evaluando en p = 0.723270
-> Iteración evaluando en p = 0.739567
-> Iteración evaluando en p = 0.739083
El valor final es: 0.739085

```

```

# función y su derivada del ejercicio 2. literal d)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return x - 0.8 - 0.2 * np.sin(x)

def df(x):
    return 1 - 0.2 * np.cos(x)

# definición de punto medio
a = 0.0
b = np.pi / 2.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")

```

```

raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-4)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-4)
print(f" El valor final es: {raiz_secante:.6f}")

```

Método de Newton

```

-> Iteración evaluando en p = 0.785398
-> Iteración evaluando en p = 0.967121
-> Iteración evaluando en p = 0.964335
El valor final es: 0.964334

```

Método de la Secante

```

-> Iteración evaluando en p = 0.000000
-> Iteración evaluando en p = 1.570796
-> Iteración evaluando en p = 0.916720
-> Iteración evaluando en p = 0.967517
-> Iteración evaluando en p = 0.964320
El valor final es: 0.964334

```

3. Use los 2 métodos en esta sección para encontrar las soluciones dentro de 10^{-5} para los siguientes problemas:

a. $3x - e^x = 0, 1 \leq x \leq 2$

b. $2x + 3\cos(x) - e^x = 0, 1 \leq x \leq 2$

```

# función y su derivada del ejercicio 3. literal a)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return 3*x - np.e ** x

def df(x):
    return 3 - np.e ** x

# definición de punto medio
a = 1.0
b = 2.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")
raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-5)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-5)
print(f" El valor final es: {raiz_secante:.6f}")

```

Método de Newton

```
-> Iteración evaluando en p = 1.500000
-> Iteración evaluando en p = 1.512358
-> Iteración evaluando en p = 1.512135
El valor final es: 1.512135
```

Método de la Secante

```
-> Iteración evaluando en p = 1.000000
-> Iteración evaluando en p = 2.000000
-> Iteración evaluando en p = 1.168615
-> Iteración evaluando en p = 1.311517
-> Iteración evaluando en p = 1.797043
-> Iteración evaluando en p = 1.436778
-> Iteración evaluando en p = 1.486766
-> Iteración evaluando en p = 1.515326
-> Iteración evaluando en p = 1.512012
-> Iteración evaluando en p = 1.512134
El valor final es: 1.512135
```

```
# función y su derivada del ejercicio 3. literal b)
def f(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return 2*x + 3* np.cos(x) - np.e ** x

def df(x):
    return 2 -3 * np.sin(x) - np.e ** x

# definición de punto medio
a = 1.0
b = 2.0

p0_newton = (a+b)/2.0

print(" Método de Newton ")
raiz_newton = optimize.newton(f, x0=p0_newton, fprime=df, tol=1e-5)
print(f" El valor final es: {raiz_newton:.6f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f, x0=a, x1=b, tol=1e-5)
print(f" El valor final es: {raiz_secante:.6f}")
```

Método de Newton

```
-> Iteración evaluando en p = 1.500000
-> Iteración evaluando en p = 1.268097
-> Iteración evaluando en p = 1.240120
-> Iteración evaluando en p = 1.239715
El valor final es: 1.239715
```

Método de la Secante

```
-> Iteración evaluando en p = 1.000000
-> Iteración evaluando en p = 2.000000
-> Iteración evaluando en p = 1.162925
-> Iteración evaluando en p = 1.216410
```

```
-> Iteración evaluando en p = 1.240684
-> Iteración evaluando en p = 1.239703
-> Iteración evaluando en p = 1.239715
El valor final es: 1.239715
```

4. El polinomio de cuarto grado

$f(x) = 230x^4 + 18x^3 + 9x^2 - 221x - 9$ tiene dos ceros reales, uno en $[-1,0]$ y el otro en $[0,1]$. Intente aproximar estos ceros dentro de 10^{-6}

- El método de la secante (use los extremos como las estimaciones iniciales)
- El método de Newton (use el punto medio como estimación inicial)

```
# función y su derivada del ejercicio 4. literal a)
def f_4(x):
    print(f" -> Iteración evaluando en p = {x:.6f}")
    return 230*x**4 + 18*x**3 + 9*x**2 - 221*x - 9

def df_4(x):
    return 920*x**3 + 54*x**2 + 18*x - 221

# definición de punto medio de [-1, 0]
a = -1.0
b = 0.0
p0_newton = (a+b)/2.0

# definición de punto medio de [0, 1]
a_1 = 0.0
b_1 = 1.0
p1_newton = (a_1+b_1)/2.0

print(" Para el intervalo [-1, 0]")
print(" Método de Newton ")
raiz_newton = optimize.newton(f_4, x0=p0_newton, fprime=df_4, tol=1e-6)
print(f" El valor final es: {raiz_newton:.8f}")

print("\n Método de la Secante ")
raiz_secante = optimize.newton(f_4, x0=a, x1=b, tol=1e-6)
print(f" El valor final es: {raiz_secante:.8f}")

print("\n Para el intervalo [0, 1]")
print(" Método de Newton ")
raiz_newton_1 = optimize.newton(f_4, x0=p1_newton, fprime=df_4, tol=1e-6)
print(f" El valor final es: {raiz_newton_1:.8f}")

print("\n Método de la Secante ")
raiz_secante_1 = optimize.newton(f_4, x0=a_1, x1=b_1, tol=1e-6)
print(f" El valor final es: {raiz_secante_1:.8f}")
```

```
Para el intervalo [-1, 0]
Método de Newton
-> Iteración evaluando en p = -0.500000
```

```
-> Iteración evaluando en p = -0.150452
-> Iteración evaluando en p = -0.041817
-> Iteración evaluando en p = -0.040659
El valor final es: -0.04065929
```

Método de la Secante

```
-> Iteración evaluando en p = -1.000000
-> Iteración evaluando en p = 0.000000
-> Iteración evaluando en p = -0.020362
-> Iteración evaluando en p = -0.030430
-> Iteración evaluando en p = -0.040667
-> Iteración evaluando en p = -0.040659
El valor final es: -0.04065929
```

Para el intervalo [0, 1]

Método de Newton

```
-> Iteración evaluando en p = 0.500000
-> Iteración evaluando en p = -0.705090
-> Iteración evaluando en p = -0.323791
-> Iteración evaluando en p = -0.064603
-> Iteración evaluando en p = -0.040686
-> Iteración evaluando en p = -0.040659
El valor final es: -0.04065929
```

Método de la Secante

```
-> Iteración evaluando en p = 0.000000
-> Iteración evaluando en p = 1.000000
-> Iteración evaluando en p = 0.250000
-> Iteración evaluando en p = 0.773763
-> Iteración evaluando en p = -1.285418
-> Iteración evaluando en p = 0.594596
-> Iteración evaluando en p = 0.394641
-> Iteración evaluando en p = -0.669318
-> Iteración evaluando en p = 0.049714
-> Iteración evaluando en p = -0.020754
-> Iteración evaluando en p = -0.040735
-> Iteración evaluando en p = -0.040659
El valor final es: -0.04065929
```

5. La función $f(x) = \tan(\pi x) - 6$ tiene cero en $\frac{1}{\pi} \arctan 6 \approx 0.447431543$. Sea $p_0 = 0$ y $p_1 = 0.48$ y use 10 iteraciones en cada uno de los siguientes métodos para aproximar esta raíz. ¿Cuál método es más eficaz y por qué?

a. Método de bisección

b. Método de Newton

c. Método de la secante

```
def f_5(x):
    return np.tan(np.pi * x) - 6

def df_5(x):
    return np.pi / (np.cos(np.pi * x)**2)
```

```

# raiz exacta
raiz_exacta = np.arctan(6) / np.pi
print(f" Raíz teórica exacta: {raiz_exacta:.9f}\n")

print("\n --- Método de la bisección ")
a = 0.0
b = 0.48
for i in range(1,11):
    p = (a + b)/ 2.0
    error = abs(p - raiz_exacta)
    print(f" Iteración {i:2d}: p = {p:.9f} | Error: {error:.9f}")
    if f_5(a) * f_5(p) < 0:
        b = p
    else:
        a = p

print("\n --- Método de Newton ")
p_newton = 0.0
for i in range(1,11):
    p_newton = p_newton - f_5(p_newton) / df_5(p_newton)
    error = abs(p_newton - raiz_exacta)
    print(f" Iteracion {i:2d}: p = {p_newton:-9f} | Error: {error:.9f}")

print("\n --- Método de la Secante ")
p0_sec = 0.0
p1_sec = 0.48
for i in range(1,11):
    p_sig = p1_sec - f_5(p1_sec) * (p1_sec - p0_sec) / (f_5(p1_sec) - f_5(p0_sec))
    error = abs(p_sig - raiz_exacta)
    print(f" Iteracion {i:2d}: p = {p_sig:-9f} | Error: {error:.9f}")
    p0_sec = p1_sec
    p1_sec = p_sig

```

Raíz teórica exacta: 0.447431543

--- Método de la bisección

```

Iteración 1: p = 0.240000000 | Error: 0.207431543
Iteración 2: p = 0.360000000 | Error: 0.087431543
Iteración 3: p = 0.420000000 | Error: 0.027431543
Iteración 4: p = 0.450000000 | Error: 0.002568457
Iteración 5: p = 0.435000000 | Error: 0.012431543
Iteración 6: p = 0.442500000 | Error: 0.004931543
Iteración 7: p = 0.446250000 | Error: 0.001181543
Iteración 8: p = 0.448125000 | Error: 0.000693457
Iteración 9: p = 0.447187500 | Error: 0.000244043
Iteración 10: p = 0.447656250 | Error: 0.000224707

```

--- Método de Newton

```

Iteracion 1: p = 1.909859 | Error: 1.462427774
Iteracion 2: p = 3.756008 | Error: 3.308576837

```

```

Iteracion 3: p = 4.906021 | Error: 4.458589760
Iteracion 4: p = 6.742795 | Error: 6.295363061
Iteracion 5: p = 7.813499 | Error: 7.366066975
Iteracion 6: p = 9.286047 | Error: 8.838615218
Iteracion 7: p = 9.871450 | Error: 9.424018390
Iteracion 8: p = 11.601410 | Error: 11.153978513
Iteracion 9: p = 11.883475 | Error: 11.436042970
Iteracion 10: p = 13.655012 | Error: 13.207580675

```

--- Método de la Secante

```

Iteracion 1: p = 0.181194 | Error: 0.266237302
Iteracion 2: p = 0.286187 | Error: 0.161244377
Iteracion 3: p = 1.091986 | Error: 0.644554563
Iteracion 4: p = -3.692297 | Error: 4.139728209
Iteracion 5: p = -22.600650 | Error: 23.048081398
Iteracion 6: p = -57.222832 | Error: 57.670264016
Iteracion 7: p = 3.538758 | Error: 3.091326602
Iteracion 8: p = -113.944405 | Error: 114.391836591
Iteracion 9: p = -195.894995 | Error: 196.342426368
Iteracion 10: p = -2989.940038 | Error: 2990.387469075

```

El método más eficaz es de la Bisección porque Newton y Secante divergen con esos puntos iniciales. Bisección garantiza convergencia siempre que f cambie de signo en el intervalo.

6. La función descrita por $f(x) = \ln(x^2 + 1) - e^{0.4x} \cos(\pi x)$ tiene un número infinito de ceros.

- Determine, dentro de 10^{-6} , el único cero negativo.
- Determine, dentro de 10^{-6} , los cuatro ceros positivos más pequeños.
- Determine una aproximación inicial razonable para encontrar el enésimo cero positivo más pequeño de f . [*Sugerencia:* Dibuje una gráfica aproximada de f]
- Use la parte c) para determinar, dentro de 10^{-6} , el vigesimoquinto cero positivo más pequeño de f .

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize

def f_6(x):
    return np.log(x**2 + 1) - np.exp(0.4 * x) * np.cos(np.pi * x)

def df_6(x):
    term1 = (2 * x) / (x**2 + 1)
    term2 = -0.4 * np.exp(0.4 * x) * np.cos(np.pi * x)
    term3 = np.pi * np.exp(0.4 * x) * np.sin(np.pi * x)
    return term1 + term2 + term3

# Generar valores de x para graficar desde -2 hasta 5

```

```

x_vals = np.linspace(-2, 10, 400)
y_vals = f_6(x_vals)

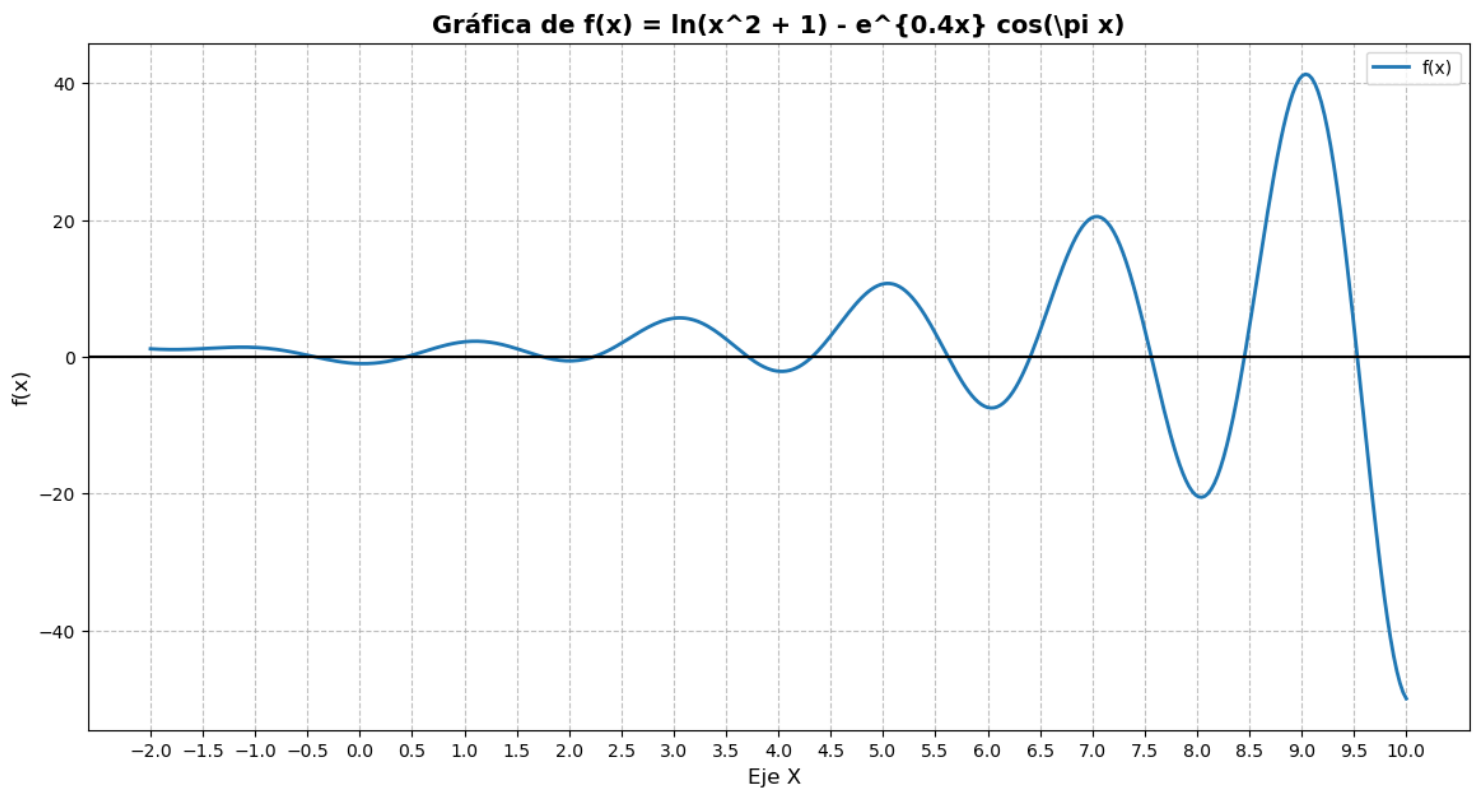
# Dibujamos la gráfica principal
plt.figure(figsize=(14, 7))
plt.plot(x_vals, y_vals, label="f(x)", color="#1f77b4", linewidth=2)
plt.axhline(0, color='black', linewidth=1.5) # Eje X

# para mostrar en intervalos de 0.5
plt.xticks(np.arange(-2, 10.5, 0.5))

plt.grid(True, linestyle='--', alpha=0.8)
plt.title("Gráfica de  $f(x) = \ln(x^2 + 1) - e^{0.4x} \cos(\pi x)$ ", fontsize=14, fontweight='bold')
plt.xlabel("Eje X ", fontsize=12)
plt.ylabel("f(x)", fontsize=12)
plt.legend()

plt.show()

```



```

import numpy as np
from scipy import optimize

print("--- Literal a) ")
po_negativo = -0.4 # estimación inicial
raiz_negativo = optimize.newton(f_6, x0=po_negativo, fprime=df_6, tol=1e-6)
print(f" Raíz negativa: {raiz_negativo:.6f}")

```

```

print("\n--- Literal b) ")
estimaciones = [0.5, 1.5, 2.5, 3.5] # para los siguientes 4 puntos
for i, p0 in enumerate(estimaciones, 1):
    raiz = optimize.newton(f_6, x0=p0, fprime=df_6, tol=1e-6)
    print(f" Cero positivo {i}: {raiz:.6f}")

print("\n--- Literal c) ")
po_positivo = 8.5 # estimación inicial
raiz = optimize.newton(f_6, x0=po_positivo, fprime=df_6, tol=1e-6)
print(f" Raíz negativa: {raiz:.6f}")

```

```

--- Literal a)
Raíz negativa: -0.434143

```

```

--- Literal b)
Cero positivo 1: 0.450657
Cero positivo 2: 1.744738
Cero positivo 3: 2.238320
Cero positivo 4: 3.709041

```

```

--- Literal c)
Raíz negativa: 8.453481

```

Para encontrar el n -ésimo cero positivo, igualamos la función a cero:

$$\ln(x^2 + 1) = e^{0.4x} \cos(\pi x)$$

A medida que x crece, el término $e^{0.4x}$ crece exponencialmente mucho más rápido que el logaritmo. Por lo tanto, para que la igualdad se mantenga, el término $\cos(\pi x)$ debe tender a cero.

Sabemos que $\cos(\theta) = 0$ cuando θ es un múltiplo impar de $\pi/2$.

Al despejar:

$$\pi x = \frac{(2n-1)\pi}{2}$$

Obtenemos la fórmula general para la aproximación inicial del n -ésimo cero:

$$p_0 = n - 0.5$$

```

print("\n--- Literal d) ")
n = 25
p0_d = n - 0.5
raiz_d = optimize.newton(f_6, x0=p0_d, fprime=df_6, tol=1e-6)
print(f" Raíz negativa: {raiz_d:.6f}")

```

```

--- Literal d)
Raíz negativa: 24.499887

```

7. La función $f(x) = x^{\frac{1}{3}}$ tiene la raíz en $x = 0$. Usando el punto de inicio de $x = 1$ y $p_0 = 5$, $p_1 = 0.5$ para el método de secante, compare los resultados de los métodos de la secante y de Newton.

```
import numpy as np
def f(x):
    return np.cbrt(x)

def df(x):
    if x == 0:
        return float('inf')
    return 1.0 / (3.0 * np.cbrt(x)**2)

p_newton = 1.0
print(" --- Método de Newton")
for i in range(1, 11):
    p_newton = p_newton - f(p_newton) / df(p_newton)
    print(f"Iteración {i:2d} p = {p_newton:12.6f}")

print("\n --- Método de la Secante")
p0_sec = 5.0
p1_sec = 0.5
for i in range(1, 11):
    denominador = f(p1_sec) - f(p0_sec)

    p_siguiente = p1_sec - f(p1_sec) * (p1_sec - p0_sec) / denominador
    print(f"Iteración {i:2d} p = {p_siguiente:12.6f}")

    p0_sec = p1_sec
    p1_sec = p_siguiente
```

```
--- Método de Newton
Iteración  1 p =  -2.000000
Iteración  2 p =   4.000000
Iteración  3 p =  -8.000000
Iteración  4 p =  16.000000
Iteración  5 p = -32.000000
Iteración  6 p =  64.000000
Iteración  7 p = -128.000000
Iteración  8 p =  256.000000
Iteración  9 p = -512.000000
Iteración 10 p = 1024.000000
```

```
--- Método de la Secante
Iteración  1 p =  -3.398012
Iteración  2 p =  -0.846851
Iteración  3 p =   3.484077
Iteración  4 p =   0.817380
Iteración  5 p =  -3.474112
Iteración  6 p =  -0.820691
Iteración  7 p =   3.475214
Iteración  8 p =   0.820324
Iteración  9 p =  -3.475091
```

Iteración 10 $p = -0.820365$

El método de Newton no puede converger cuando $x = 0$, porque tiende a infinito. El método de la Secante diverge, pero oscila entre los valores ± 3.47 y ± 0.81 . Ninguno de los dos métodos sirve para raíces donde la derivada no existe o se anula.